

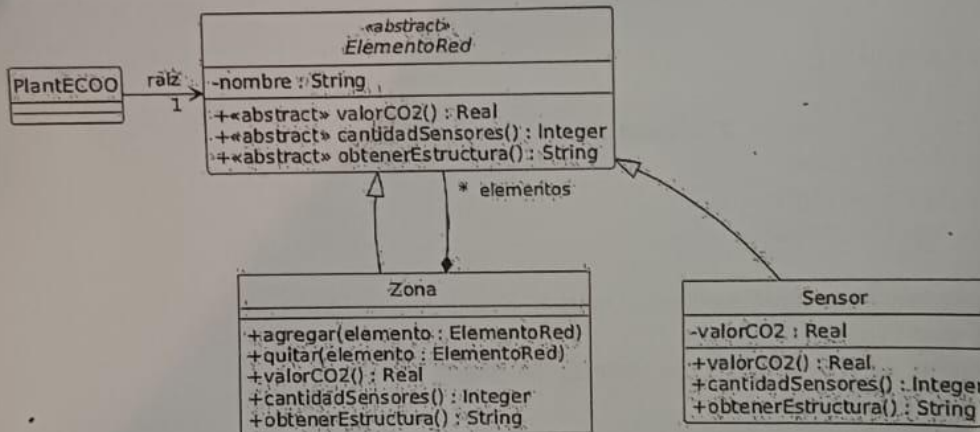
La empresa ambiental **PlantECOO** posee una red de monitoreo desplegada en distintos puntos de la ciudad de La Plata que permite registrar mediciones de la concentración de dióxido de carbono, con el objetivo de detectar zonas con potencial de riesgo ambiental y asistir en la toma de decisiones. Actualmente el sistema modela la red de monitoreo a partir de dos tipos de elementos:

- **Sensores:** dispositivos físicos individuales capaces de registrar una medición actual de CO2.
- **Zonas:** agrupaciones de elementos de monitoreo. Una zona puede contener tanto sensores individuales como otras zonas.

La primera versión del sistema ya permite trabajar con esta estructura para obtener información de cada sector monitoreado. En particular, para cualquier elemento de la red se puede:

- Obtener el valor de CO2. El sensor retorna su medición mientras que la zona retorna el promedio.
- Conocer el número total de sensores involucrados.
- Consultar la estructura de zonas y subzonas.

El diagrama UML del sistema actual es el siguiente:



Actividades - parte 1:

1. Indique el nombre del patrón de diseño que se utiliza.
2. Explique con sus palabras (entre 2 y 5 renglones) cuál es el objetivo del patrón indicado en el inciso anterior.

La empresa ambiental PlantECOO decidió avanzar con una nueva funcionalidad para intervenir en la gestión ambiental. Cuando un usuario observa valores de contaminación (CO2) preocupantes debe poder activar un mecanismo de mitigación, como por ejemplo: (i) reducción temporal del tránsito, (ii) Instalación de barreras vegetales, o (iii) inspección técnica. Pueden aparecer nuevos mecanismos en el futuro. Se desea incorporar a la red la posibilidad de definir un mecanismo que debe ser utilizado para todos los elementos. El mismo se define por el área técnica y puede cambiarse cuando el área lo necesite.

Actividades - parte 2:

3. Indique el nombre del patrón que aplicaría para incorporar la posibilidad de definir el mecanismo de mitigación.
4. Explique con sus palabras (entre 2 y 5 renglones) cuál es el objetivo del patrón indicado en el inciso anterior.
5. Realice el diagrama UML incorporando el patrón de diseño previamente definido para que la clase PlantECOO colabore con su solución. Indique los roles en las clases utilizando estereotipos.

6. Codifique las clases que involucren el patrón conforme a sus respuestas previas. Implemente únicamente el código del mecanismo "Reducción temporal del tránsito". Tenga presente que el mecanismo de reducción es muy complejo e intervienen muchas clases, sin embargo, para efectos de este ejercicio (para simplificar el código esperado), considere que la implementación se reduce a lo siguiente:

```
System.out.println("activando reducción temporal del tránsito")
```

7. Escriba el código para definir por defecto el mecanismo de mitigación "Reducción temporal del tránsito". Recuerde considerar que se podría cambiar el mecanismo de mitigación.
8. Escriba en la clase correspondiente el método `activarMitigacion()`, en donde se desencadena el mecanismo de mitigación.

Refactoring

OO2 - Parcial 06/06/2026

El siguiente código implementa una clase que gestiona la reproducción y el cobro de contenidos (Películas y Música). La aplicación ha crecido desordenadamente y mezcla la lógica de negocio con el formato de salida.

```
1. public class Contenido {
2.     public String titulo;
3.     public int tipo; // 1: Película, 2: Música
4.     private double duracion;
5.     public String streamingUrl;
6.     private int reproducciones;
7.
8.     public void gestionarReproduccion(int megas, String usuarioTipo) {
9.         if (tipo == 1) {
10.             System.out.println("Iniciando Película: " + titulo);
11.             if (megas > 1024) {
12.                 System.out.println("Usando " + megas / 1024 + " GB de ancho de banda.");
13.             } else {
14.                 System.out.println("Usando " + megas + " MB de ancho de banda.");
15.             }
16.         } else if (tipo == 2) {
17.             System.out.println("Iniciando Audio: " + titulo + " con duración: " + duracion);
18.         }
19.         reproducciones++;
20.         double precioBase = 100.0;
21.         double total = 0;
22.         if (usuarioTipo.equals("PREMIUM")) {
23.             total = precioBase;
24.         } else {
25.             total = precioBase * 1.21 + 50.0;
26.         }
27.         System.out.println("Cobro registrado: " + total);
28.     }
29.     public String getDetalleParaLog() {
30.         return "ID-77" + "|" + titulo + "|" + streamingUrl + "|" + duracion;
31.     }
32. }
33. }
```

(Refactoring) Actividades - parte 1:

1.- Marcar V o F (Verdadero o Falso)

#	Ubicación	Afirmación sobre Code Smell / Diseño	V / F
1	Líneas 2 y 5.	Existe Public Field , rompiendo el encapsulamiento.	V
2	Línea 11	El uso de <code>int</code> megas es un caso de Primitive Obsession .	
3	Líneas 9 a 18	Se observa Switch Statement que debería resolverse con Polimorfismo.	
4	Línea 19	El incremento de la variable <code>reproducciones</code> es un Magic Number .	
5	Línea 20	La variable <code>precioBase</code> es Dead Code .	
6	Línea 31	Se observa Feature Envy .	F

Actividades - parte 2:

Ahora considere el siguiente código:

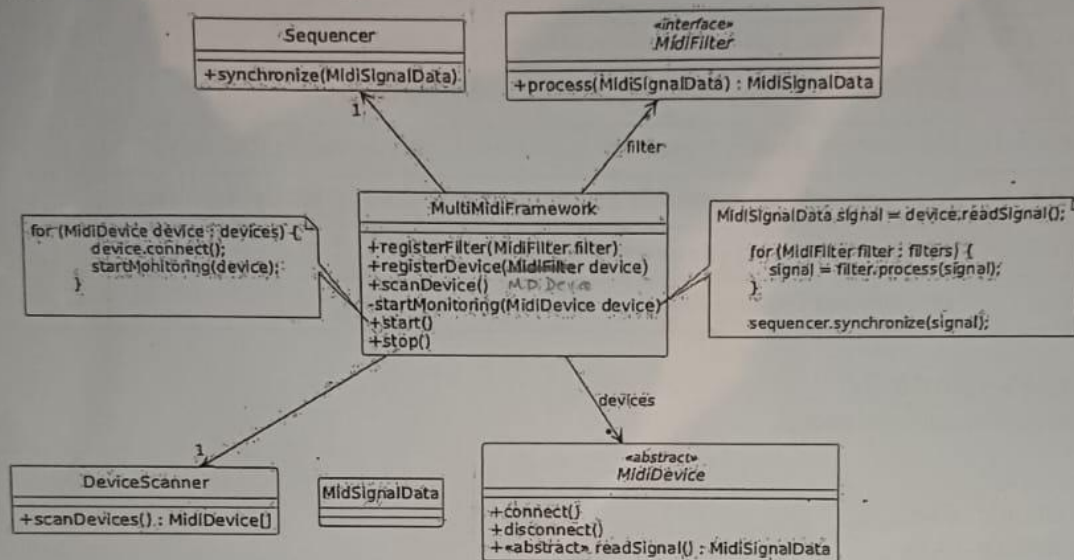
```
class Triangulo {  
    private double base;  
    private double altura;  
    public Triangulo(double base, double altura) {  
        this.base = base;  
        this.altura = altura;  
    }  
    public double getBase() {  
        return base;  
    }  
    public double getAltura() {  
        return altura;  
    }  
}  
  
class Calculadora {  
    public double calcularSuperficie(Triangulo triangulo) {  
        return (triangulo.getBase() * triangulo.getAltura()) / 2;  
    }  
}
```

- 2.- Indique el mal olor que se encuentra en el método `calcularSuperficie()`
- 3.- Indique el refactoring que lo corrige.
- 4.- Aplique el refactoring (vuelva a escribir el código).

Frameworks

OO2 - Parcial 06/06/2026

El protocolo MIDI es un estándar de comunicación entre instrumentos musicales digitales (como sintetizadores y baterías eléctricas) y computadoras u otros dispositivos de audio. El framework "MultiMIDI" permite construir aplicaciones de grabación y procesamiento MIDI.



Para utilizar el framework, la aplicación debe instanciar y registrar todos los dispositivos MIDI que va a utilizar. Opcionalmente, puede registrar también filtros de procesamiento de sonido. Luego debe llamar al método start() para conectarse con los dispositivos y comenzar la grabación. Para terminar, el método stop() finaliza el procesamiento y desconecta los dispositivos.

Ejemplo de uso, donde YamahaKeyboard, RolandDrumPad y EchoFilter no son parte del framework:

```

public class Aplicacion {

    public static void main(String[] args) {
        MultiMidiFramework framework = new MultiMidiFramework();
        framework.registerDevice(new YamahaKeyboard());
        framework.registerDevice(new RolandDrumPad());
        framework.registerFilter(new EchoFilter());
        framework.start();
    }
}
    
```

Tareas:

1. ¿Considera que el framework "MultiMIDI" es WhiteBox o BlackBox? Justifique en no más de 2 renglones.
2. Para cada uno de los ítems, indique si es hot spot o frozen spot.
 - a. El método readSignal() de MidiDevice
 - b. El método start() de MultiMidiFramework
 - c. El método registerDevice() de MultiMidiFramework
 - d. El método process() de MidiFilter
3. ¿Hay inversión de control en el método readSignal() de MidiDevice? ¿Y en synchronize() de Sequencer?
4. ¿Qué cambios se requieren realizar para que el framework soporte el nuevo dispositivo llamado BluetoothMidiKeyboard? Explique brevemente en no más de tres renglones.

Comise Luciana

Patrones A Refactor: A Fromulak, A

1/3

25470/4

AUFMUTU ALVARO

Patrones - Parte 1

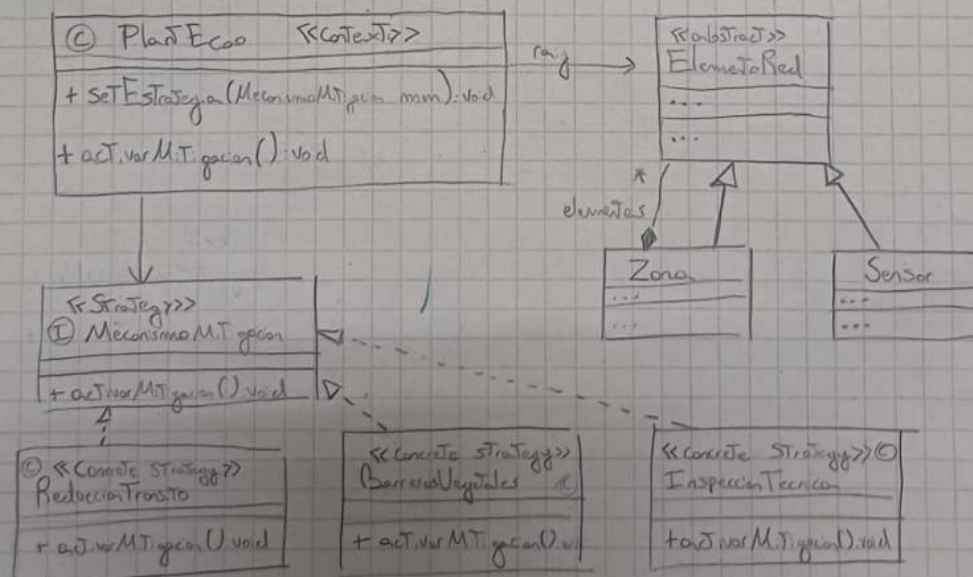
- 1- El patrón utilizado es el Composite /
- 2- El objetivo del patrón es Tratar Tanto Sensores como zonas de igual manera, haciendo que contengan los mismos mensajes. En este caso los sensores serian las hojas, las zonas las compuestas y ElementoReal el componente. las Zonas en este caso pueden contener sensores como otras Zonas

Parte 2

- 3- Usara un Strategy con las estrategias concretas ReduccionTrasito, BarreraVigilales e InspeccionTecnica
- 4- La idea del patrón es poder cambiar dinamicamente la manera de realizar una tarea, la manera de activar de cada mecanismo es distinta pero todos saben realizar la accion activarMitigacion

- encapsula formulas de algoritmos -

5-



NOTA

```
6- public interface MecanismoMitigacion {
    void activarMitigacion();
}
```

```
public class ReduccionTransito implements MecanismoMitigacion {
    @Override
    public void activarMitigacion() {
        System.out.println("activando reduccion Temporal del Transito");
    }
}
```

// El contexto lo define en el punto 7

```
7- public class PlanEcoo {
    private MecanismoMitigacion mecanismo;
    public PlanEcoo() {
        mecanismo = new ReduccionTemporal();
    }
    public void setEstrategia(MecanismoMitigacion mm) {
        mecanismo = mm;
    }
}
```

```
8- public void activarMitigacion() {
    mecanismo.activarMitigacion();
}
```

// pertenece a la clase PlanEcoo, la idea es que el usuario físico aprete el botón, ahí PlanEcoo delega la activación a su estrategia

25470/4

AUFMUTH ALVARO

HORA

2/3

FECHA

Refactoring

#

1	Lineas 2 y 5	V	1
2	Linea 11	F	1
3	Lineas 9 a 18	V	1
4	Linea 19	F	1
5	Linea 20	F	1
6	Linea 31	F	1

2- El code simil que se encuentra aca es Feature envy, ya que el mismo Triangulo deberan saber calcular su ^{superficie} ya que es quien tiene la informacion

3- El refactoring que lo corrige es Move Method de calcularSuperficie() hacia Triangulo

4 Hago el move method, copio el metodo Tal cual en la clase Triangulo, determino que no se requieren parametros ya que Triangulo tiene Todo lo necesario para el calculo. Delego la llamada de calculadora al metodo nuevo en Triangulo

```
class Triangulo {
```

```
    private double base;
    private double altura;
    public Triangulo (double base, double altura) {
        This.altura = altura;
        This.base = base;
    }
```

```
    public double getBase() { return This.base; }
    public double getAltura() { return This.altura; }
```

```
    public double calcularSuperficie () {
```

```
        return (This.getBase() * This.getAltura()) / 2;
```

```
    }
```

```
// sigue aTras
```

NOTA

2547 / 1

```
class Calculadora {  
    public double calcularSuperficie(Triangulo Triangulo) {  
        return Triangulo.calcularSuperficie();  
    }  
}
```

class Calculadora {

public double Calculadora(int, int) {

25470/4

AUFMUTU ALVARO

Frameworks

1- MULTIMIDI es un Framework de bajo nivel, ya que su estructura es por composición. No se debe entender el funcionamiento interno del Framework sino "estructura" los dispositivos y clases a utilizar.

2-

a) readSignal() es un hot spot

b) start() es un FrozenSpot

c) registerDevice() es un FrozenSpot

d) process() es un hot spot

3- En el método readSignal() hay inversión de control, en el método synchronize() no

4- No se requieren cambios en el Framework, solo hace falta crear la clase Bluetooth/MidiKeyboard haciendo que herede la clase MidiDevice, implemente obligatoriamente el método readSignal().

25470/4

Partes - Parte 1

1- El primer dígito es el

El dígito del primer
número que está
seguido, los que los
casos pueden estar

3/3

Parte 2

3- Usar un Stream

Implementación

4- La clase

saber real

5-